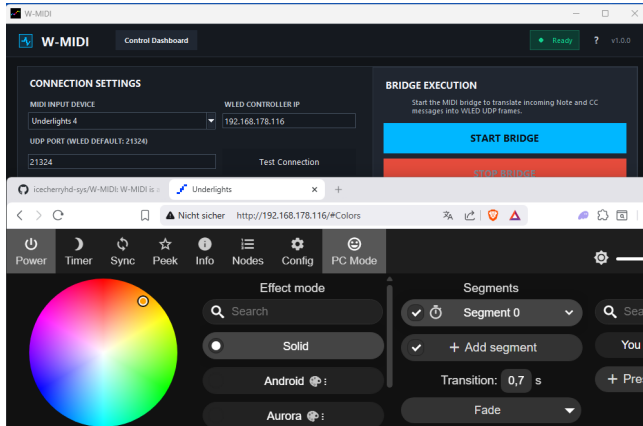


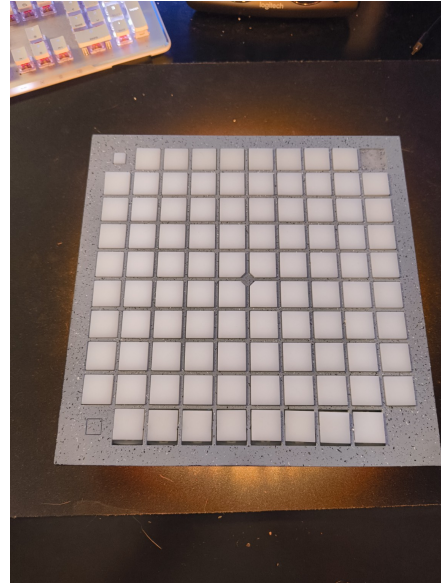
W-MIDI Tutorial Guide

Expanded edition with hardware build photos, WLED disclaimer, feature explanations, and first-use workflow

This guide explains how to build a simple underlight setup and how to use W-MIDI to control it. It is written for people who are new to the project and want a practical step-by-step overview.



W-MIDI controls WLED over the network.



A finished underlight panel can sit underneath a controller or pad surface.

What this guide covers

- Hardware requirements and example costs.
- Important WLED notes and where to learn more about WLED setup.
- The physical underlight structure and electronics placement.
- Downloading W-MIDI and preparing the software workflow.
- Setting up loopMIDI, WLED, Ableton Live, and W-MIDI.
- An extra feature overview page for the main W-MIDI interface sections.
- Troubleshooting the most common issues.

2. Before you begin: WLED note and example costs

There are many WLED tutorials online, and this guide is **not** a full WLED setup manual. The focus here is W-MIDI and a compact example underlight build. For detailed WLED topics such as flashing, LED wiring, power calculations, effect behavior, and larger installations, use the WLED documentation and external WLED tutorials as your main reference.

Recommended WLED starting points

- **Official WLED getting started guide:** <https://kno.wled.ge/basics/getting-started/>
- **Official WLED install guide:** <https://kno.wled.ge/basics/install-binary/>
- **Example tutorial topic to search for:** “WLED ESP32 beginner tutorial” or “WLED power injection tutorial”.

Responsibility note

Power design, current draw, wiring safety, LED selection, and any larger WLED setup are the responsibility of the builder. Always check what your own hardware needs. The example in this guide is a small and lightweight setup only.

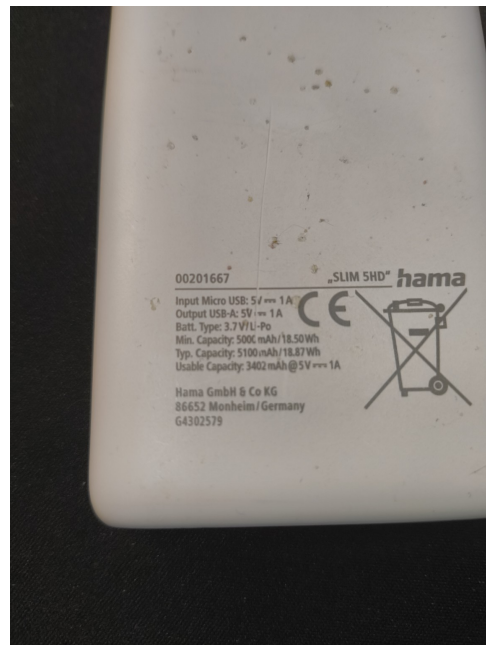
Typical example cost factors

- **ESP32:** roughly €5–15 depending on the board and seller.
- **Power bank:** roughly €10–25 depending on capacity and brand.
- **LED strip:** roughly €10–30 or more depending on strip length, LED density, and type.
- **Cables / connectors / small accessories:** often another €3–10.

For the power bank shown in this example, the label lists an output of **5 V / 1 A**. That is treated here as a **minimum example requirement** for this small build, and it is sufficient in this specific case. Larger or brighter WLED installations may require significantly more current.



Example hardware set: ESP32 board, short USB cable, and power bank.



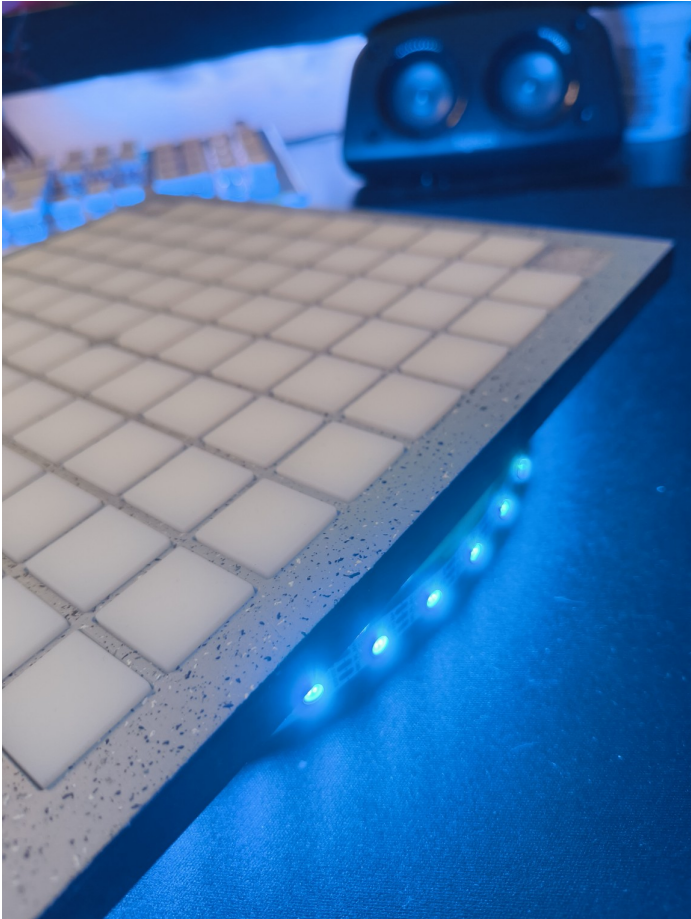
Power bank label. The important value in this example is the output rating: 5 V / 1 A.

3. What W-MIDI does

W-MIDI is a realtime bridge between MIDI and WLED. MIDI notes or controller data are received by W-MIDI and translated into UDP packets for WLED. This makes it possible to drive a WLED LED setup from Ableton Live or another MIDI source.

Signal flow

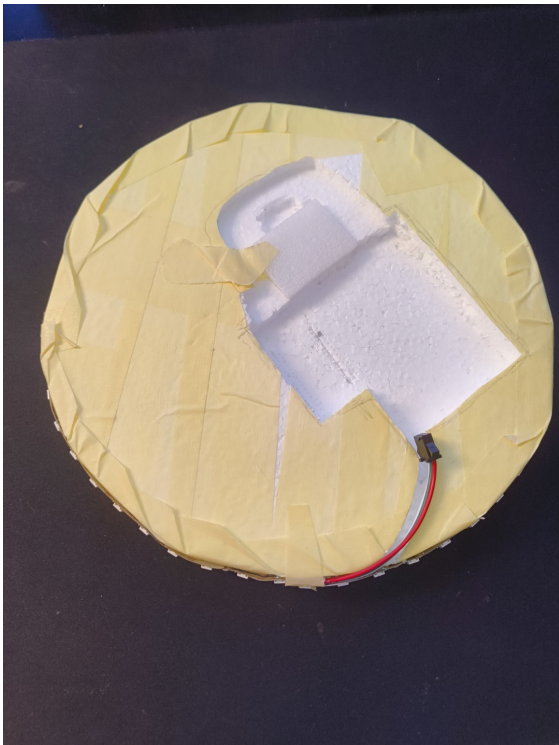
Ableton Live → loopMIDI virtual port → W-MIDI → WLED on ESP32 → LED strip / underlight



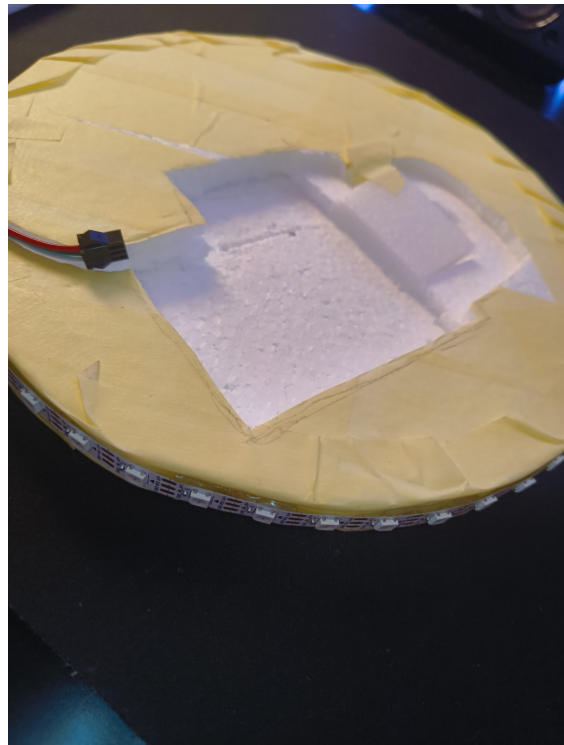
Example result: when the setup is working, the LEDs under the controller react and create a visible glow.

4. Underlight structure

The underlight structure sits underneath the controller and carries the LED strip around the outer edge. A cavity in the middle provides space for the ESP32 and the power bank.



Top view of the underlight structure before the electronics are inserted.



Angled view showing the cavity for the electronics and the LED strip mounted around the edge.

What to notice

- The LED strip runs around the perimeter of the structure.
- The center cavity creates room for the ESP32 and a compact power source.
- The cable path should remain accessible so the build stays easy to connect and maintain.
- A compact layout helps keep the finished structure flat enough to sit underneath the controller.

5. Electronics placed inside

After preparing the structure, place the ESP32 and the power bank inside the center cavity. The photos below show the internal arrangement and a closer look at the wiring.



Full internal layout: ESP32, power bank, cable routing, and LED connection.



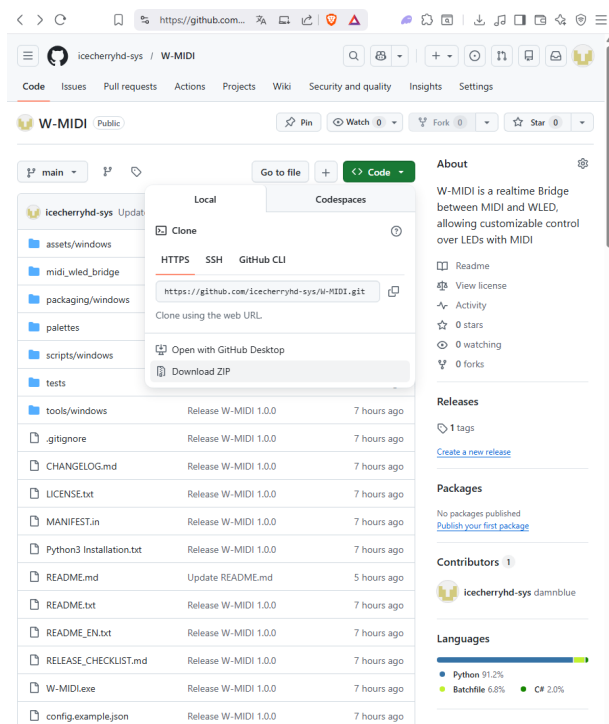
Closer view of the installed ESP32 and power bank while connected.

Practical advice

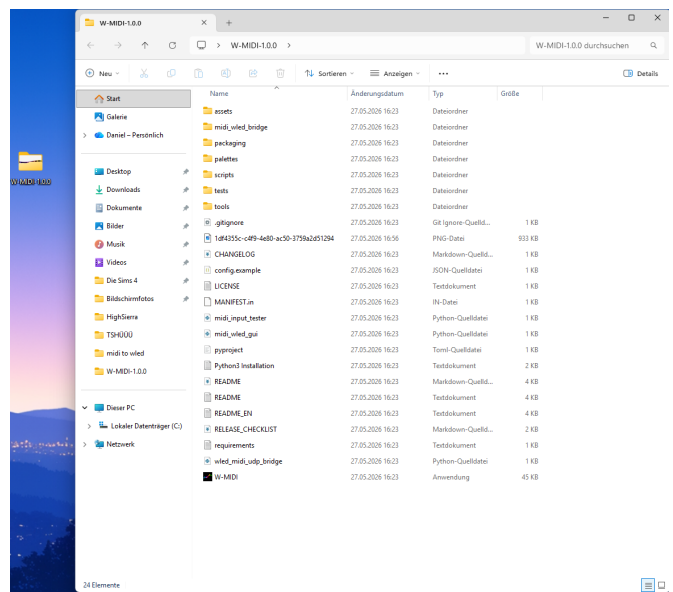
- Make sure the ESP32 is powered securely before starting WLED or W-MIDI.
- Keep the cable path tidy so the parts do not move around during use.
- If the structure will be moved often, secure the parts firmly to protect the wiring.

6. Download and unpack W-MIDI

Download the repository or release package and extract it into a folder that is easy to access.



GitHub page for W-MIDI. Use the **Code** menu to download the project as a ZIP, or use a prepared release folder.



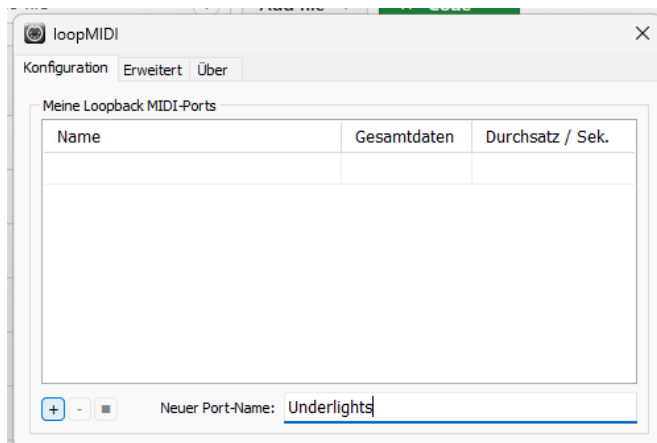
The extracted project folder. The main desktop application is **W-MIDI.exe**.

Key files

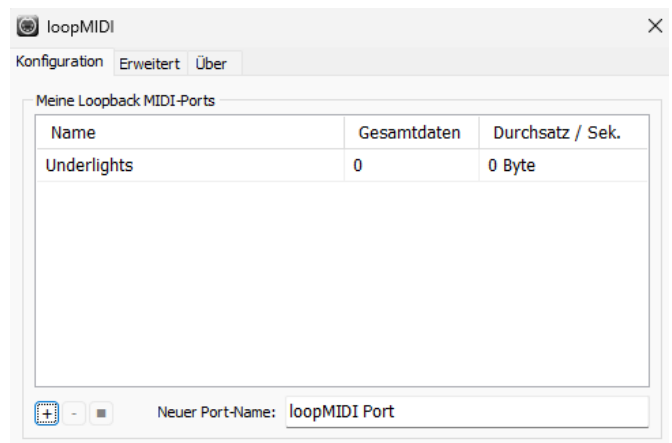
- **W-MIDI.exe** – the main desktop application.
- **README_EN.txt** – a quick reference file.
- **palettes/velocity_palette.txt** – a ready-made velocity palette.
- **requirements.txt** – only needed if you want to run the project from Python instead of the EXE.

7. Create a virtual MIDI port with loopMIDI

W-MIDI needs a MIDI input device. If you want to send MIDI from Ableton Live, the easiest method is to create a virtual loopback port in loopMIDI. In this guide, the port is called **Underlights**.



Enter a new port name such as **Underlights** and create it with the plus button.

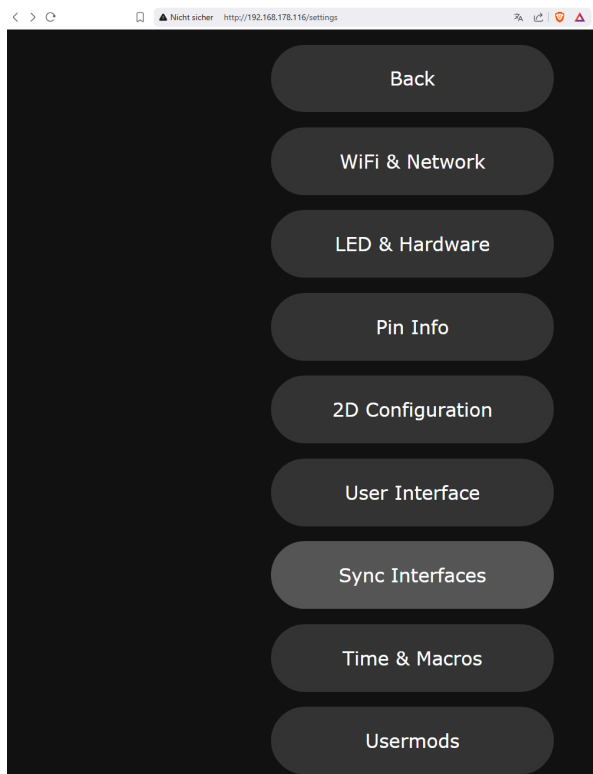


The port appears in the list once it has been created.

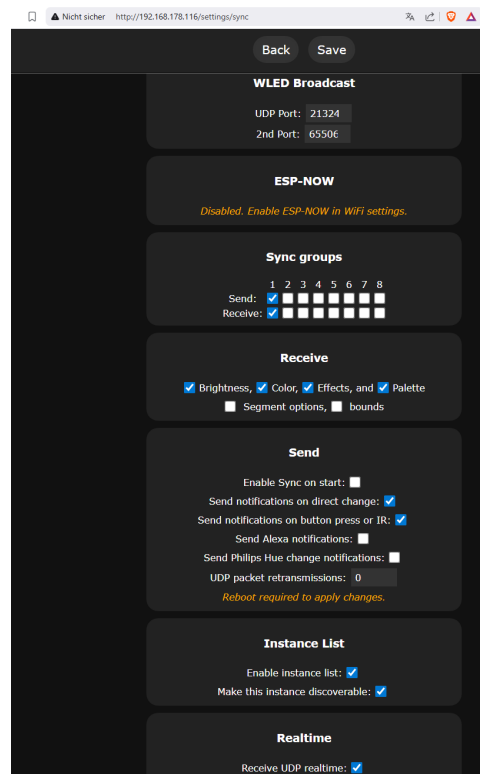
- Open loopMIDI.
- Enter a port name in the name field.
- Create the port.
- Leave loopMIDI running while you use Ableton and W-MIDI.
- If W-MIDI was already open, press **Reload Ports** later so the new port appears.

8. Configure WLED on the ESP32

Before W-MIDI can control the LEDs, WLED must be reachable on the network and configured to accept realtime UDP data.

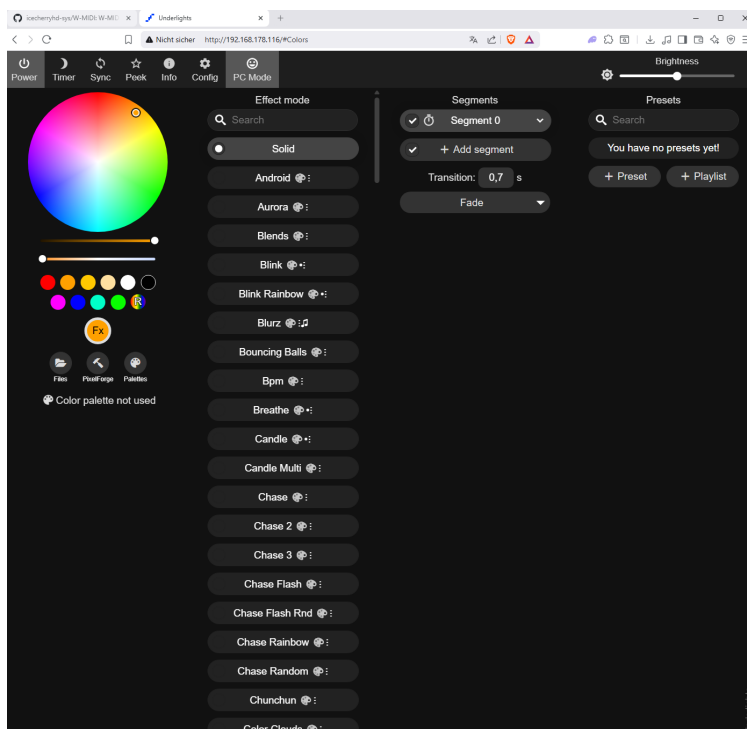


Open the WLED **Config** menu and navigate to the relevant settings pages.



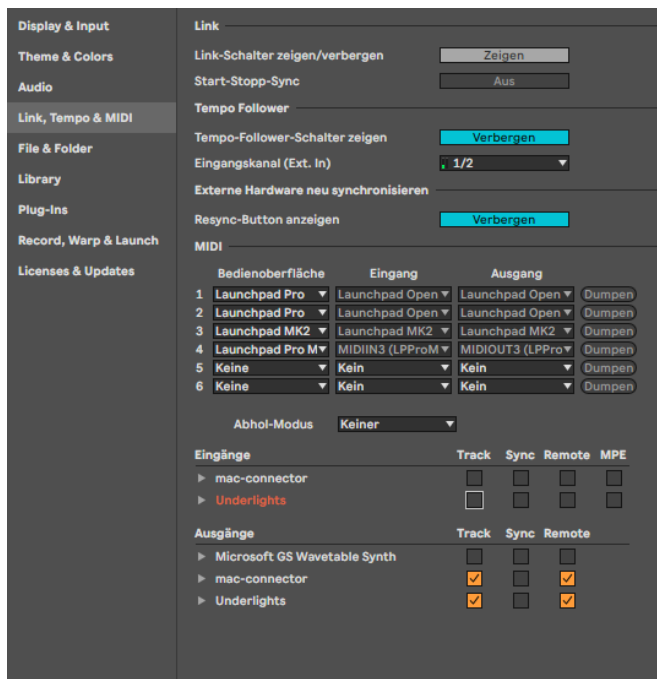
On the **Sync Interfaces** page, verify the UDP settings. A common default is port **21324**.

- Know the WLED device IP address.
- Use the same UDP port in WLED and W-MIDI.
- Save any changes you make in the WLED settings.
- If WLED is not reachable in the browser, fix the network connection before moving on.

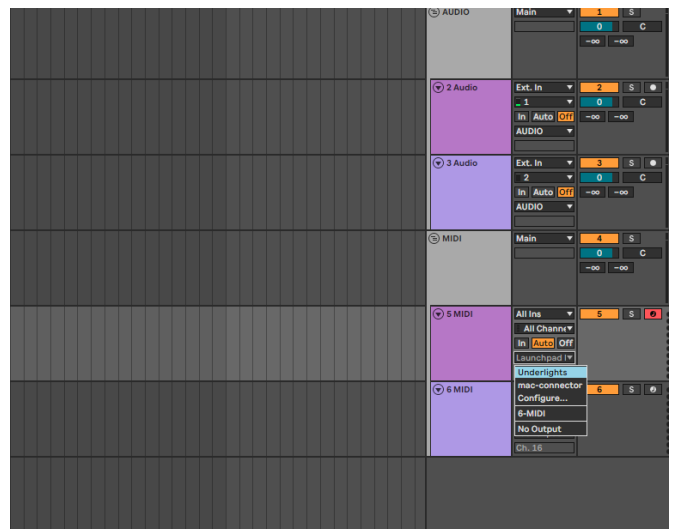


9. Configure Ableton Live

Enable the loopMIDI port in Ableton so MIDI notes can be routed to W-MIDI. First, enable the port in the MIDI preferences, then set a track output to that port.



Ableton **Link, Tempo & MIDI** preferences. Enable the **Underlights** output so tracks can send MIDI to it.

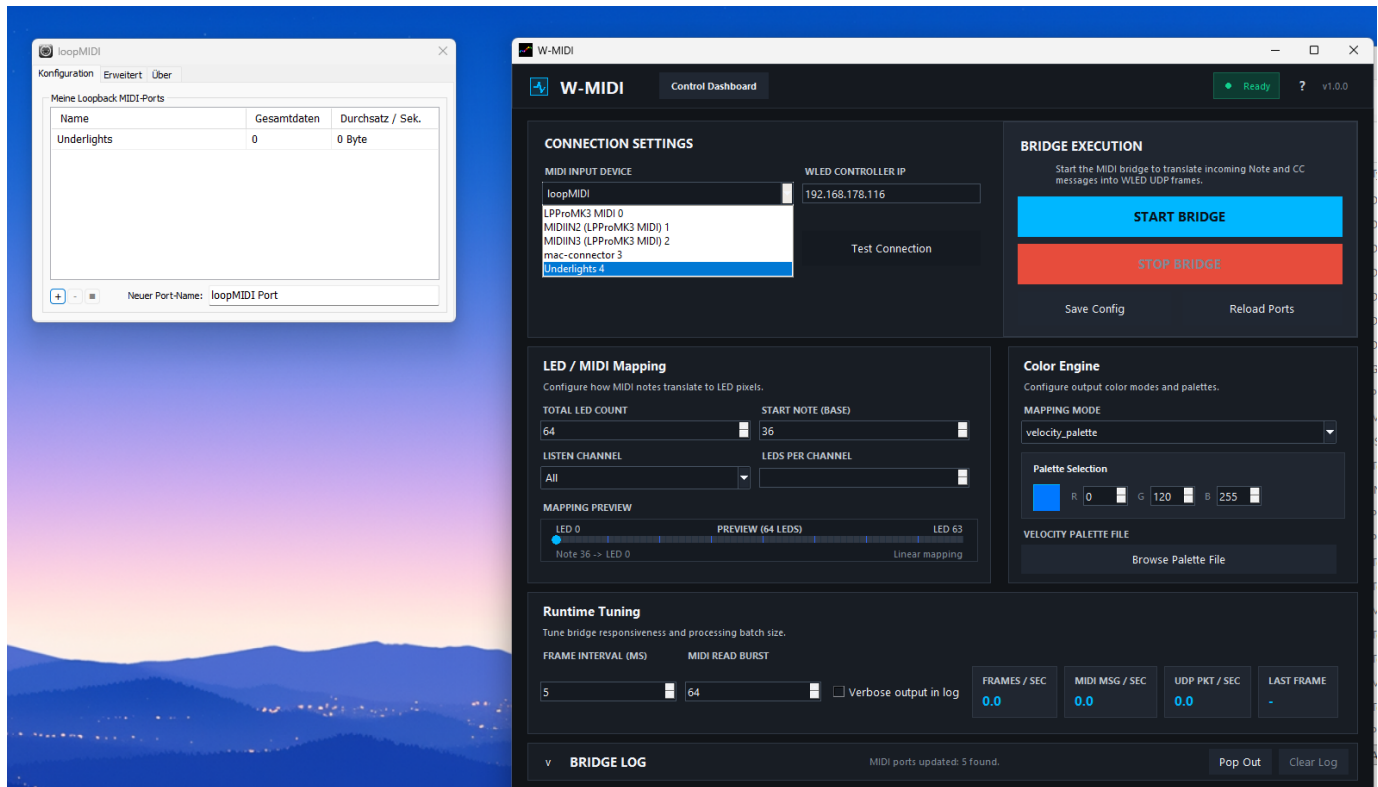


Track routing view in Ableton. Set the track output to the **Underlights** port.

- Open **Preferences** → **Link, Tempo & MIDI**.
- Find the **Underlights** port under outputs.
- Enable at least **Track** for that output.
- Set the MIDI track output to **Underlights**.
- Send notes, clips, or controller data from that track.

10. Configure W-MIDI

Now open **W-MIDI.exe**. Choose the MIDI input device, enter the WLED IP address, and set the core mapping and performance options.



W-MIDI desktop dashboard with connection settings, LED / MIDI mapping, color engine, runtime tuning, and the bridge log.

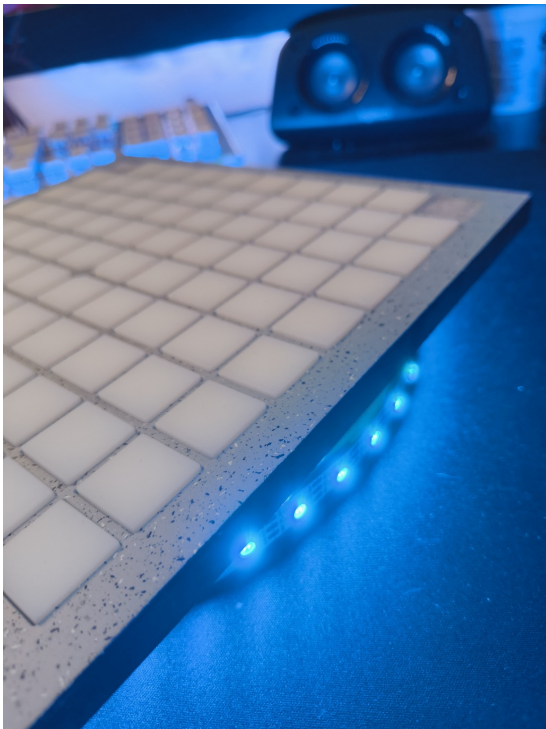
Recommended first settings

- **MIDI Input Device:** the same port you created in loopMIDI.
- **WLED Controller IP:** the IP address of your WLED device.
- **UDP Port:** usually **21324**.
- **Total LED Count:** set this to match your LED installation.
- **Start Note (Base):** choose the note that should map to LED 0.
- **Listen Channel:** use **All** unless a single MIDI channel is required.
- **Mapping Mode / Color Engine:** **velocity_palette** is a useful default.
- **Frame Interval:** **5 ms** is a safe starting point.
- **MIDI Read Burst:** **64** is a safe starting point.

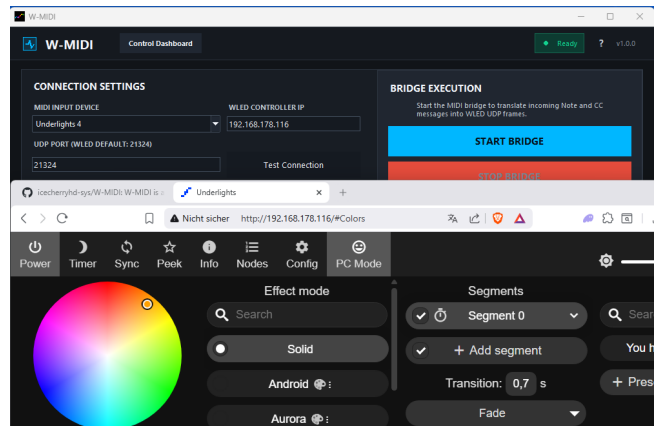
After editing the settings, click **Save Config**. If you change the setup later, it is best to stop the bridge first, save the changes, and then start the bridge again.

11. Test the connection and use the setup

Before playing a full MIDI performance, use the connection tools to confirm that the system works. The blue underglow photo is a good example of a successful visible reaction.

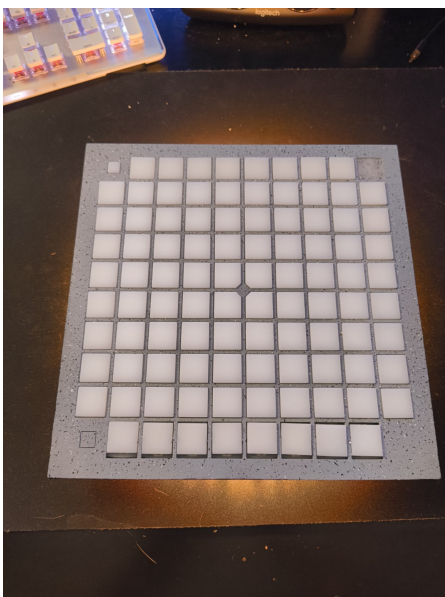


*Example of the LEDs glowing blue. This is a useful visual reference after pressing **Test Connection**.*

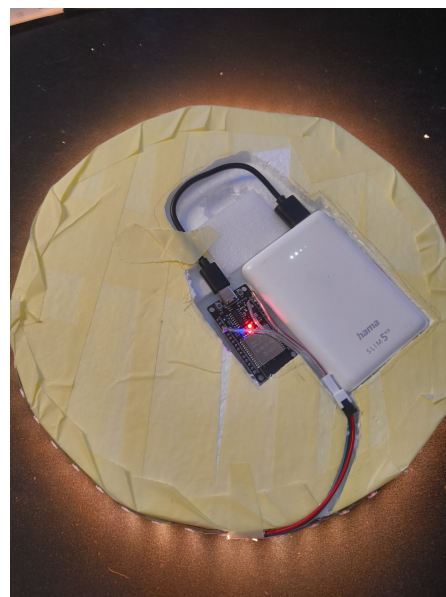


W-MIDI running together with the WLED interface. Once the connection is correct, the live response can be monitored here.

- Press **Test Connection** in W-MIDI.
- Watch the LEDs for a visible reaction.
- Look at the W-MIDI log and live statistics for activity.
- If nothing happens, recheck the WLED IP address and UDP port first.
- If the connection test works but MIDI does not, the problem is usually in the MIDI routing between Ableton, loopMIDI, and W-MIDI.



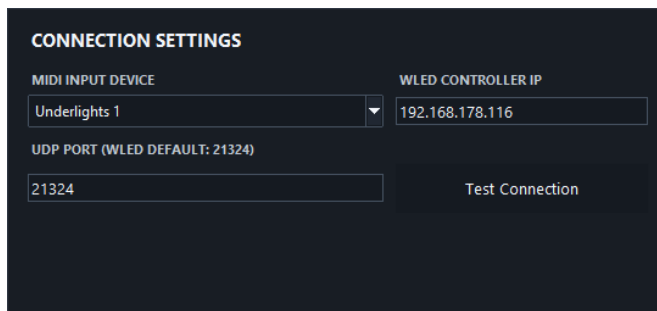
Top view of the finished controller / panel placement.



Internal arrangement with the ESP32 and power bank connected inside the structure.

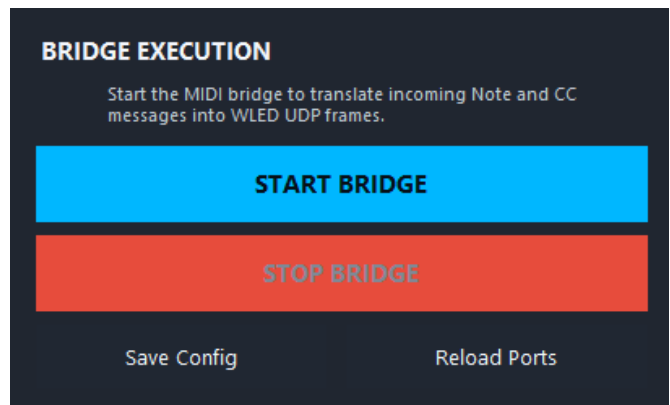
12. Extra feature overview: core W-MIDI sections

The screenshots below highlight the main sections of the W-MIDI interface. This page explains what each section is for, what the most important fields mean, and how they affect the bridge behavior.



The screenshot shows the 'CONNECTION SETTINGS' section. It has a dark background with white text. There are two input fields: 'MIDI INPUT DEVICE' with a dropdown menu showing 'Underlights 1' and a 'WLED CONTROLLER IP' field with the value '192.168.178.116'. Below these is a 'UDP PORT (WLED DEFAULT: 21324)' field with the value '21324'. To the right of the UDP port field is a 'Test Connection' button.

Connection Settings: choose the MIDI input device, set the WLED IP address, define the UDP port, and test the connection.



The screenshot shows the 'BRIDGE EXECUTION' section. It has a dark background with white text. At the top, it says 'Start the MIDI bridge to translate incoming Note and CC messages into WLED UDP frames.' Below this are two large buttons: a blue 'START BRIDGE' button and a red 'STOP BRIDGE' button. At the bottom are two smaller buttons: 'Save Config' and 'Reload Ports'.

Bridge Execution: start or stop the bridge, save the configuration, and reload the available MIDI ports.

Connection Settings

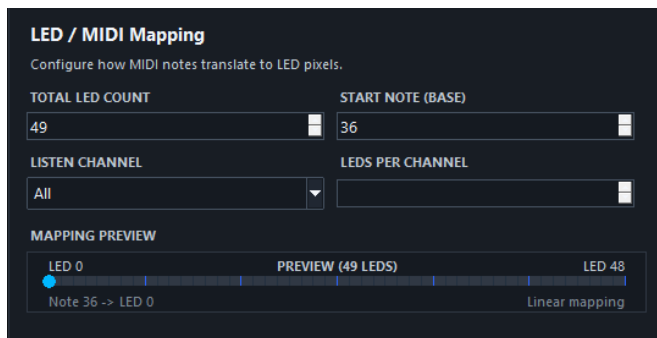
- **MIDI Input Device** selects the MIDI source that W-MIDI listens to. This should match the loopMIDI port or hardware port you want to use.
- **WLED Controller IP** is the network address of the WLED device running on the ESP32.
- **UDP Port** must match the WLED realtime UDP port, typically **21324**.
- **Test Connection** sends a simple test to confirm that WLED can be reached.

Bridge Execution

- **Start Bridge** begins realtime MIDI-to-WLED translation.
- **Stop Bridge** stops the live bridge without closing the application.
- **Save Config** stores the current settings for later use.
- **Reload Ports** refreshes the list of available MIDI input devices, which is helpful after creating a new loopMIDI port.

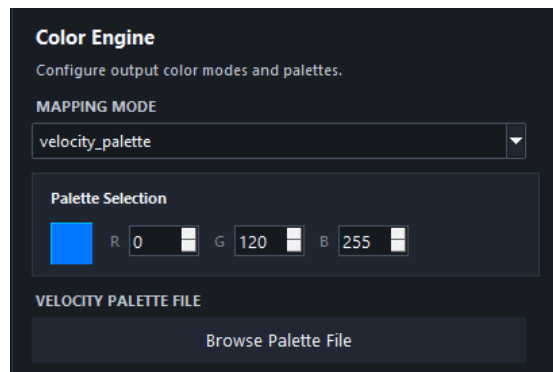
13. Extra feature overview: mapping, color, and runtime

These sections define how notes are mapped to LEDs, how colors are generated, and how responsive the bridge feels in practice.



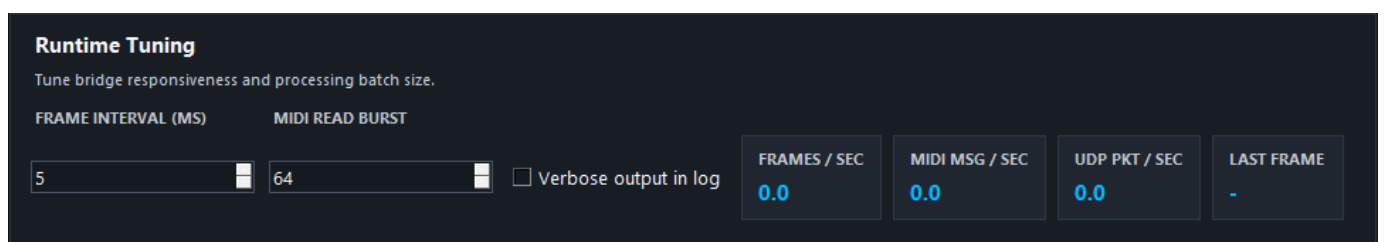
The 'LED / MIDI Mapping' interface is a dark-themed control panel. It features a title 'LED / MIDI Mapping' and a subtitle 'Configure how MIDI notes translate to LED pixels.' Below this are four input fields: 'TOTAL LED COUNT' (set to 49), 'START NOTE (BASE)' (set to 36), 'LISTEN CHANNEL' (set to 'All'), and 'LEDS PER CHANNEL' (empty). A 'MAPPING PREVIEW' section shows a horizontal bar from 'LED 0' to 'LED 48' with a blue dot at 'LED 0' and the text 'Note 36 -> LED 0' and 'Linear mapping'.

LED / MIDI Mapping: define the LED count, starting note, channel filter, and note-to-LED relationship.



The 'Color Engine' interface is a dark-themed control panel. It features a title 'Color Engine' and a subtitle 'Configure output color modes and palettes.' Below this is a 'MAPPING MODE' dropdown menu set to 'velocity_palette'. A 'Palette Selection' section shows a blue square and RGB values: R: 0, G: 120, B: 255. A 'VELOCITY PALETTE FILE' section has a 'Browse Palette File' button.

Color Engine: choose the color generation mode and, if needed, a velocity palette file.



The 'Runtime Tuning' interface is a dark-themed control panel. It features a title 'Runtime Tuning' and a subtitle 'Tune bridge responsiveness and processing batch size.' Below this are two input fields: 'FRAME INTERVAL (MS)' (set to 5) and 'MIDI READ BURST' (set to 64). A checkbox 'Verbose output in log' is unchecked. On the right, four status boxes show 'FRAMES / SEC' (0.0), 'MIDI MSG / SEC' (0.0), 'UDP PKT / SEC' (0.0), and 'LAST FRAME' (-).

Runtime Tuning: adjust bridge responsiveness and processing behavior, and monitor basic live statistics.

LED / MIDI Mapping

- **Total LED Count** defines how many LED pixels are available in the installation.
- **Start Note (Base)** defines which MIDI note maps to LED 0.
- **Listen Channel** can listen to all channels or only a specific MIDI channel.
- **LEDs per Channel** can be used when a channel-based mapping layout is needed.
- **Mapping Preview** shows how notes are distributed across the LED range.

Color Engine

- **Mapping Mode** controls how incoming note data is translated into color behavior.
- **velocity_palette** is a useful mode when note velocity should influence the output color.
- **Palette Selection** provides a quick visual reference for the active color values.
- **Velocity Palette File** lets you load a custom palette file from disk.

Runtime Tuning

- **Frame Interval (ms)** affects how often frames are sent. Lower values can feel more responsive but may increase load.
- **MIDI Read Burst** controls how many MIDI messages are processed per read cycle.
- **Verbose output in log** adds more detailed log information for debugging.
- **Frames / Sec**, **MIDI Msg / Sec**, **UDP Pkt / Sec**, and **Last Frame** help monitor what the bridge is doing live.

14. Troubleshooting

If the setup does not work immediately, use the checklist below. Most problems come from power, network configuration, or MIDI routing.

- **No power:** confirm that the ESP32 is powered and that the power source is strong enough for the LED load.
- **No WLED page in the browser:** solve the Wi-Fi or network issue first.
- **No reaction in W-MIDI:** make sure the correct loopMIDI port is selected and visible.
- **Test Connection does nothing:** verify the WLED IP address and UDP port.
- **MIDI arrives but the mapping feels wrong:** check LED count, base note, and MIDI channel settings.
- **Large or bright LED setup:** a small power bank may not be enough. Increase the power supply as needed and follow WLED power guidance.

Optional command-line tools

- `py -3 -m pip install -r requirements.txt` – install Python dependencies.
- `py -3 -m midi_wled_bridge.midi_tester --list-only` – list available MIDI ports.
- `py -3 -m midi_wled_bridge.gui` – start the GUI from Python.
- `py -3 -m midi_wled_bridge.cli --wled-ip 192.168.1.100 --midi-port "Underlights" --led-count 49 --base-note 36 --color-mode velocity_palette --velocity-palette-file "palettes/velocity_palette.txt"` – start the bridge from the command line.

You now have a complete beginner-friendly reference for the W-MIDI workflow, from the hardware build to the first LED test and the main software feature overview.